

CHAPTER 5 – LARGE AND FAST: EXPLOITING MEMORY HIERARCHY PATTERSSON/HENNESSY

Luiz Paulo de Assis Barbosa

5.1 – Introdução

2

- Desde os dias iniciais da computação programadores desejam quantidades ilimitadas de memória.
- Veremos, no estudo deste capítulo, maneiras de criar a ilusão de uma memória ilimitada por meio de uma hierarquia de memória.
- Criar uma hierarquia de memória é possível, devido aos programas não acessarem a totalidade de seu código ou dados, ao mesmo tempo, com igual probabilidade.

5.1 – Introdução

3

- Esta característica dos programas é melhor expressa por meio do **princípio da localidade**, que diz:
 - ▣ os programas acessam uma porção relativamente pequena do seu espaço de endereços em dado intervalo de tempo.
- Existem dois tipos de localidade:
 - ▣ **Localidade temporal** – Se um item é referenciado, ele tende a ser referenciado novamente em breve.
 - ▣ **Localidade espacial** – Se um item é referenciado, itens cujos endereços são próximos tendem a ser referenciados em breve.

5.1 – Introdução

4

- Uma **hierarquia de memória** consiste em múltiplos níveis de memória de diferentes velocidades e tamanhos (capacidade de armazenamento).
- As memórias mais rápidas têm custo por bit maior que as mais lentas e por isso utiliza-se porções menores dessas memórias.
- Existem três principais tecnologias usadas nos sistemas de memória hoje em dia:
 - ▣ DRAM, SRAM e discos magnéticos.

5.1 – Introdução

- DRAM (Dinamic Random Access Memory) – Memória RAM dinâmica, necessita que um processo de *refresh* dos dados armazenados seja realizado periodicamente.
 - Sem esse processo os dados seriam perdidos, mesmo sem a interrupção da energia.
 - Grande número de unidades de armazenamento por unidade de área.
- SRAM (Static Random Access Memory) – Memória RAM estática, não necessita de *refresh* dos dados armazenados.
 - Pequeno número de unidades de armazenamento por unidade de área.
 - OBS: tanto a DRAM quanto a SRAM são voláteis, ou seja perdem os dados armazenados quando a energia é interrompida.

5.1 – Introdução

6

- Discos Magnéticos – A informação é guardada por meio da orientação dos dipolos magnéticos de pequenas porções da superfície do disco.
 - Este tipo de memória é não-volátil.

5.1 – Introdução

7

- A meta ao se criar uma hierarquia de memória é:
 - ▣ apresentar ao usuário grande quantidade de memória barata (DRAM), porém com velocidade de acesso próxima a das memórias mais caras (SRAM).
- Para isso, usa-se menor quantidade de memórias rápidas e caras próximas ao processador (cache) e maior quantidade de memórias baratas e lentas em níveis mais altos.

5.1 – Introdução

8

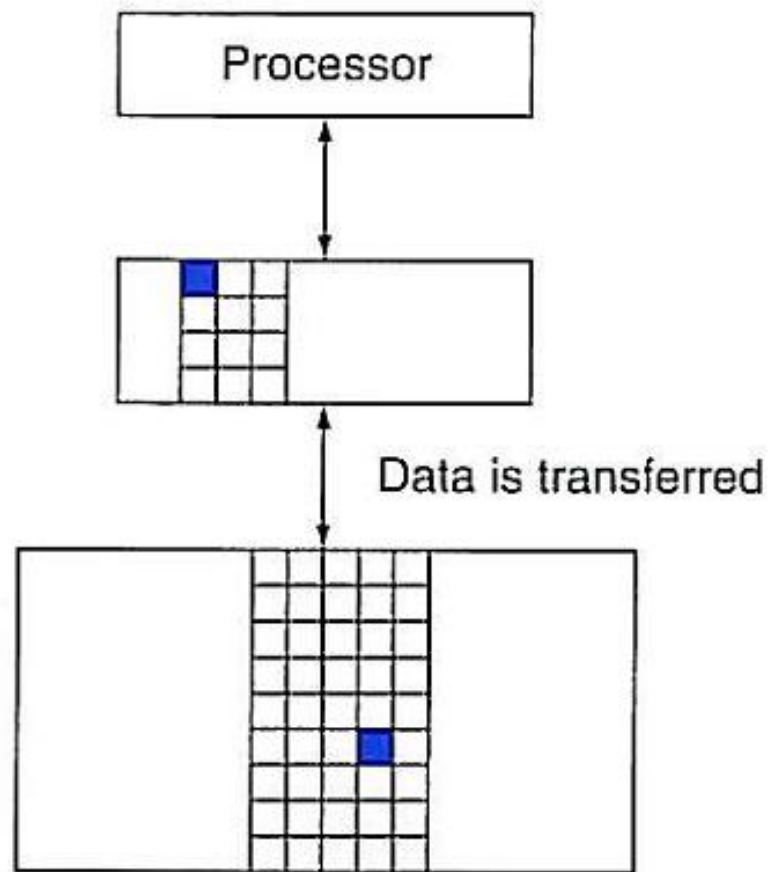
Speed	Processor	Size	Cost (\$/bit)	Current technology
Fastest	Memory	Smallest	Highest	SRAM
	Memory			DRAM
Slowest	Memory	Biggest	Lowest	Magnetic disk

5.1 – Introdução

- A distribuição dos dados na hierarquia de memória também é hierárquica: um nível próximo ao processador (nível alto) guarda um subconjunto dos dados armazenados nos níveis mais distantes (mais baixos).
- A totalidade dos dados é armazenada no nível mais distante (mais baixo).
- Uma hierarquia de memória pode ser composta por vários níveis, mas os dados são copiados entre 2 níveis adjacentes apenas, em um dado momento.

5.1 – Introdução

10



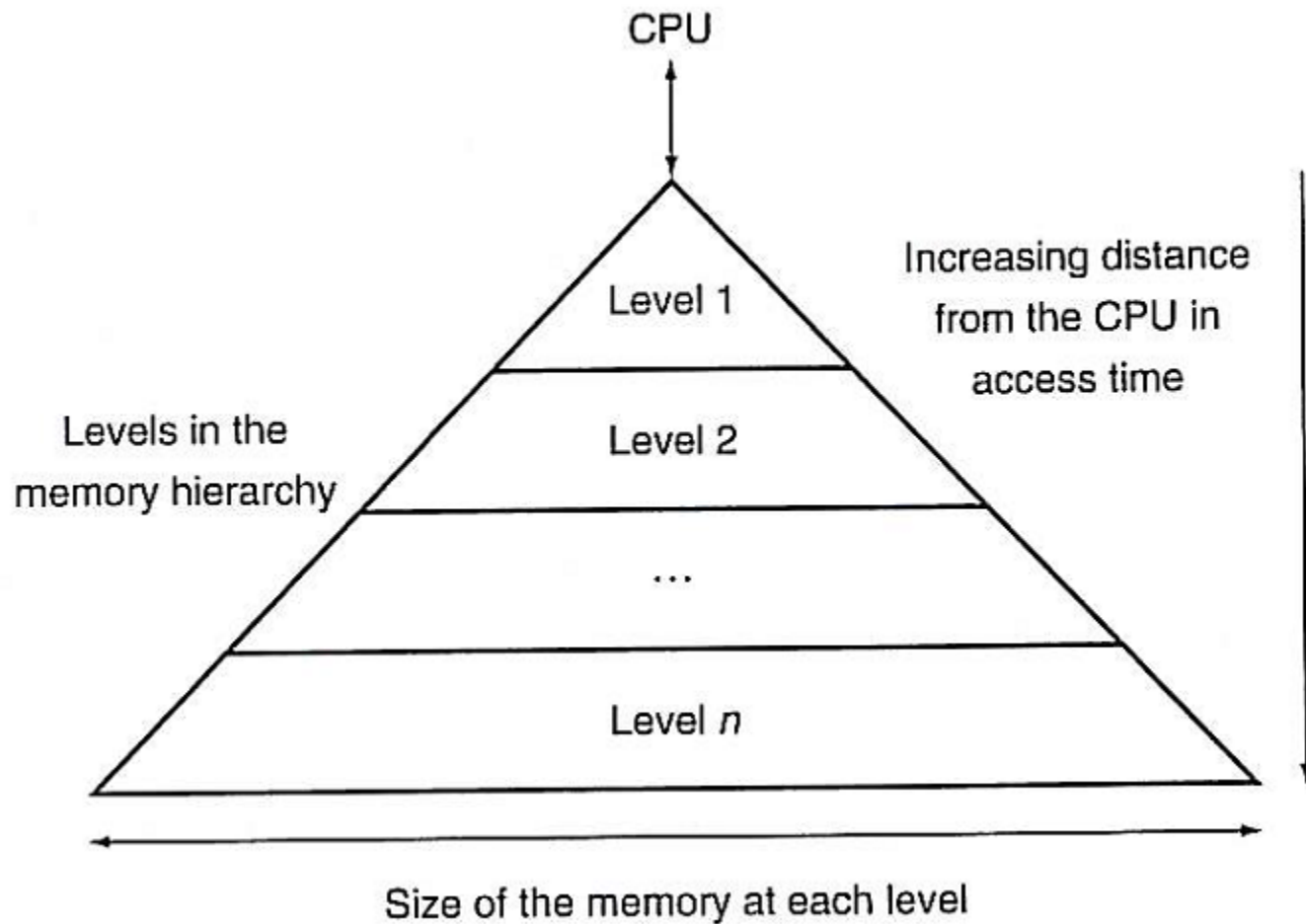
5.1 – Introdução

11

- Bloco ou linha
 - ▣ É a menor unidade de informação que pode estar ou não presente na cache.
- Taxa de acertos
 - ▣ É a fração dos acessos à memória encontrados em um dado nível da hierarquia de memória.
- Taxa de erros
 - ▣ É a fração dos acessos à memória não encontrados em um dado nível da hierarquia de memória.

5.1 – Introdução

12



5.2 – O básico sobre caches

13

- Considere uma cache na qual cada bloco consiste de apenas uma palavra.

X_4
X_1
X_{n-2}
X_{n-1}
X_2
X_3

a. Before the reference to X_n

X_4
X_1
X_{n-2}
X_{n-1}
X_2
X_n
X_3

b. After the reference to X_n

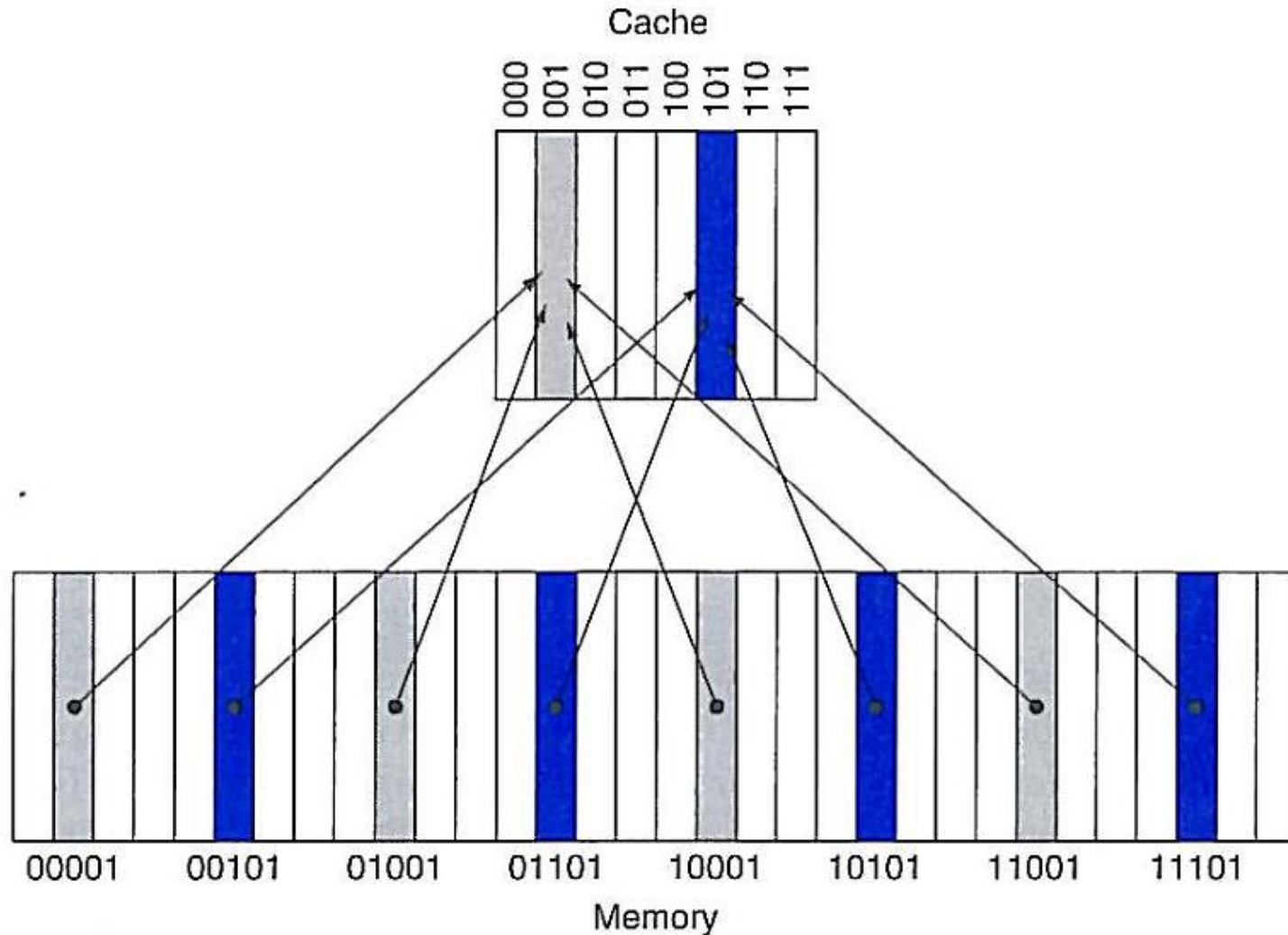
5.2 – O básico sobre caches

14

- No cenário descrito anteriormente:
 - ▣ Como sabemos se um dado está na cache?
 - ▣ Se ele está, como encontrá-lo?
- A maneira mais simples de atribuir uma posição da cache para cada palavra na memória é atribuir a posição da cache com base no endereço da palavra de memória.
- Esta estrutura de cache é conhecida por **mapeamento direto**.
 - ▣ **Índice = Endereço do bloco % Número de blocos da cache.**
 - ▣ **% (módulo, resto da divisão)**

5.2 – O básico sobre caches

15



5.2 – O básico sobre caches

16

- Ssequência de acessos à memória, considerando a cache inicialmente vazia.

Decimal address of reference	Binary address of reference	Hit or miss in cache	Assigned cache block (where found or placed)
22	10110_{two}	miss (7.6b)	$(10110_{\text{two}} \bmod 8) = 110_{\text{two}}$
26	11010_{two}	miss (7.6c)	$(11010_{\text{two}} \bmod 8) = 010_{\text{two}}$
22	10110_{two}	hit	$(10110_{\text{two}} \bmod 8) = 110_{\text{two}}$
26	11010_{two}	hit	$(11010_{\text{two}} \bmod 8) = 010_{\text{two}}$
16	10000_{two}	miss (7.6d)	$(10000_{\text{two}} \bmod 8) = 000_{\text{two}}$
3	00011_{two}	miss (7.6e)	$(00011_{\text{two}} \bmod 8) = 011_{\text{two}}$
16	10000_{two}	hit	$(10000_{\text{two}} \bmod 8) = 000_{\text{two}}$
18	10010_{two}	miss (7.6f)	$(10010_{\text{two}} \bmod 8) = 010_{\text{two}}$
16	10000_{two}	hit	$(10000_{\text{two}} \bmod 8) = 000_{\text{two}}$

5.2 – O básico sobre caches

17

- Estado inicial da cache. (Vazia)

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	N		
111	N		

5.2 – O básico sobre caches

18

- Estado da cahe após o erro na solicitação do endereço 10110_2

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

5.2 – O básico sobre caches

19

- Estado da cahe após o erro na solicitação do endereço 11010_2

Index	V	Tag	Data
000	N		
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

5.2 – O básico sobre caches

20

- Estado da cahe após o erro na solicitação do endereço 10000_2

Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

5.2 – O básico sobre caches

21

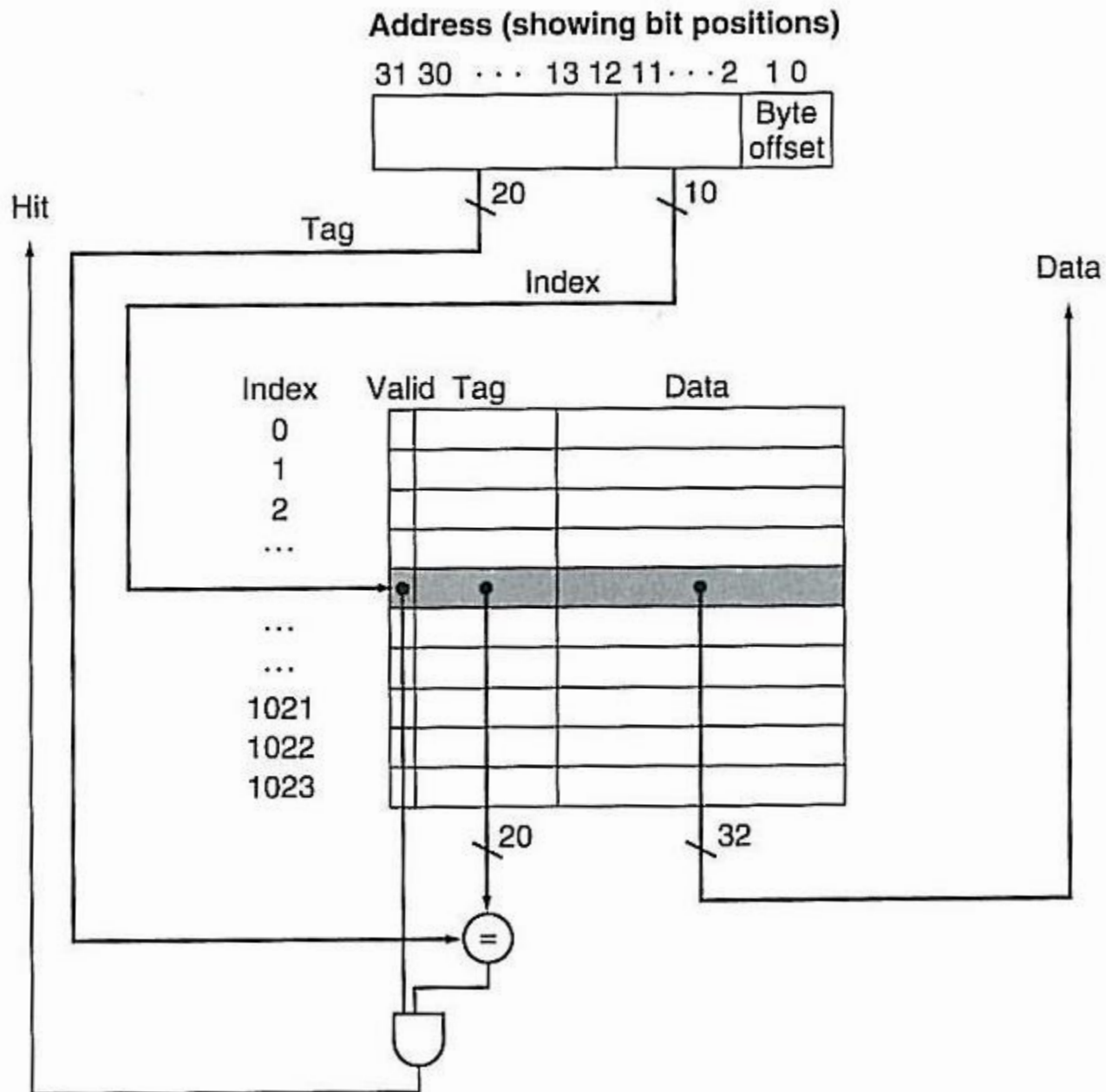
- Estado da cahe após o erro na solicitação do endereço 00011_2

Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	Y	00_{two}	Memory (00011_{two})
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

5.2 – O básico sobre caches

- Estado da cahe após o erro na solicitação do endereço 10010_2

Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
010	Y	10_{two}	Memory (10010_{two})
011	Y	00_{two}	Memory (00011_{two})
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		



5.2 – O básico sobre caches

24

- Se temos 32 bits para endereço de byte (memória principal).
- Para uma cache, usando mapeamento direto, temos:
 - ▣ O tamanho da cache é 2^n blocos, logo n bits são usados para o campo índice.
 - ▣ O tamanho do bloco é 2^m palavras (cada uma 4 bytes logo 2^{m+2} bytes), assim m bits são usados para endereçar uma palavra em um bloco.
 - ▣ Os 2 bits menos significativos são usados para achar um byte dentro de uma palavra.
 - ▣ Com isso o campo tag tem $32 - (n + m + 2)$ bits.

5.2 – O básico sobre caches

25

- O número total de bits em uma cache que usa mapeamento direto é:

$$2^n \times (\text{tamanho de bloco} + \text{tamanho da tag} + \text{tamanho do campo de validade})$$

- Assim, como o tamanho de um bloco é:
 - ▣ 2^{m+5} bits, pois, $2^m \times 32$ bits (uma palavra) $= 2^m \times 2^5 = 2^{m+5}$.
 - ▣ E precisamos de 1 bit de validade.
- Temos:
 - ▣ $2^n \times (2^m \times 32 + (32 - n - m - 2) + 1)$
 - ▣ $2^n \times [(2^m \times 32) + 31 - n - m]$ bits
- Porém, a convenção é considerar apenas o tamanho dos dados na cache para nomeá-la, assim a cache do slide anterior é uma cache de 4KB (4 Bytes X 1024 linhas).

5.2 – O básico sobre caches

26

- Exemplo: Número de bits em uma cache
- Qual o total de bits requeridos por uma cache de 16KB de dados e blocos de 4 palavras, assumindo um endereço de 32 bits?
- Solução:
 - Sabendo que cada palavras tem 4 bytes, na região de dados cabem $16\text{KB} / 4 \text{ bytes} = 4\text{K}$ palavras (2^{12}).
 - Como o tamanho de um bloco é 4 palavras (2^2 , logo $m=2$), temos que o número de blocos nessa cache é $4\text{K} \text{ palavras} / 4 \text{ palavras por bloco} = 1024 \text{ blocos}$ (2^{10} , logo $n = 10$).

5.2 – O básico sobre caches

27

- ▣ Cada bloco tem 4 palavras x 32 bits por palavra = 128 bits de dados
- ▣ Mais a tag que tem $[32 - (10 + 2 + 2)] = (32 - 10 - 2 - 2) = 18$ bits
- ▣ Mais o bit de validade, assim:
- ▣ Por bloco ou linha = $128 + 18 + 1 = 147$ bits.
- ▣ Total = 147 bits por bloco x 1024 blocos = 147Kbits.

5.2 – O básico sobre caches

28

- Exemplo: Mapeando um endereço para uma cache de múltiplas palavras por bloco.
- Considere uma cache com 64 blocos e um tamanho de bloco de 16 bytes. Para qual número de bloco (ou para qual linha) o endereço de byte 1200 é mapeado?
- Solução:
 - ▣ Como o mapeamento é direto, temos:
 - ▣ Número do bloco (linha) = (endereço de bloco) módulo (número de blocos da cache)

5.2 – O básico sobre caches

29

- ▣ O endereço de bloco é encontrado dividindo o endereço de byte pelo número de bytes por bloco, $1200/16 = 75$.
- ▣ Como a cache tem 64 blocos, temos $75 \text{ módulo } 64 = 11$, ou seja, 75 dividido por 64 tem quociente 1 e resto 11.
- ▣ Assim, 1200 será mapeado para a linha 11 ou com índice 11.
- ▣ Na verdade esse bloco contém todos os endereços entre:
 - 1200
 - e $1200 + \text{número de bytes por bloco} - 1 = 1200 + 16 - 1 = 1215$.
 - Se considerarmos 32 bits de endereço, temos:
 - 4M blocos da memória principal (MP) que mapeiam para essa linha.
 - A MP teria, para 32 bits de endereço:
 - 256M blocos no total,
 - Ou $256M \times 16 \text{ bytes (4 palavras)} = 4096MB = 4GB$.

5.2 – O básico sobre caches

30

- Vamos ver o que aconteceu em termos de bits no exemplo anterior (assumindo 32 bits de endereço):

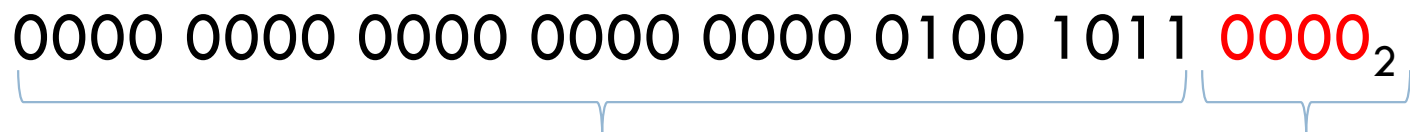
- ▣ $1200_{10} = 00004B0_{16}$

- ▣ Em binário:

0000 0000 0000 0000 0000 0100 1011 0000₂

- ▣ Para distinguirmos entre os 16 bytes (2^4) de um bloco precisamos de 4 bits, os menos significativos, assim:

0000 0000 0000 0000 0000 0100 1011 0000₂



75₁₀
end. de bloco

0₁₀
end. de byte

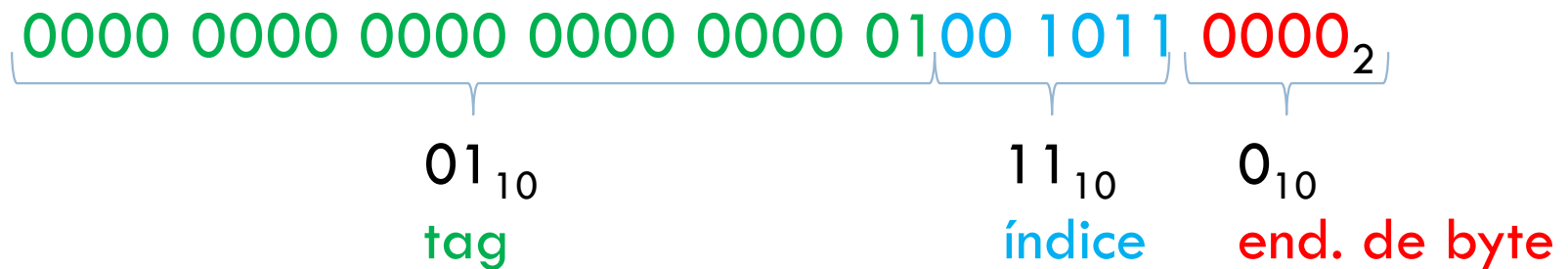
5.2 – O básico sobre caches

31

□ continuando:

- Como temos 64 blocos (2^6) e o campo de índice é usado para distinguí-los, temos que esse campo deve ter 6 bits.

■ logo:

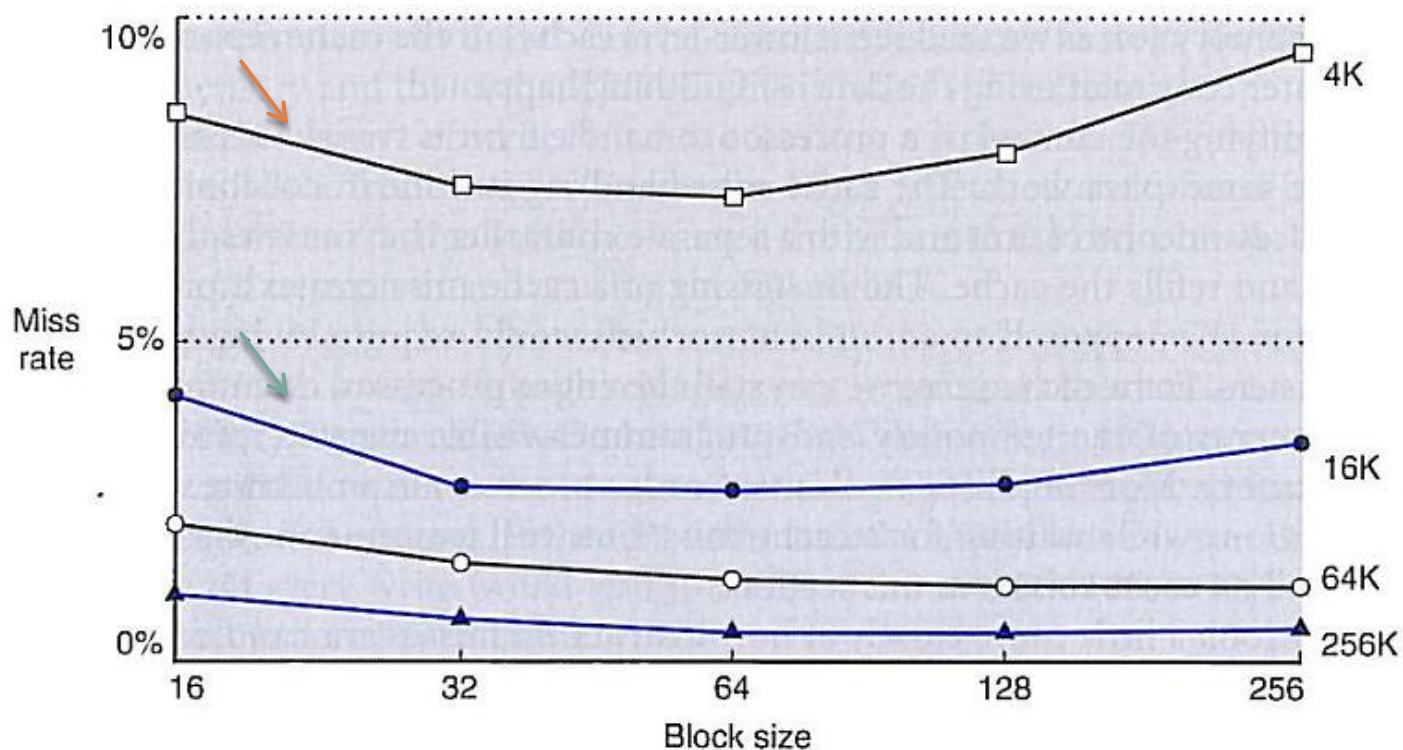


- Observe que o campo de tag tem 22 bits como esperado já que 4M blocos (2^{22}) mapeiam na linha com índice 11.

5.2 – O básico sobre caches

32

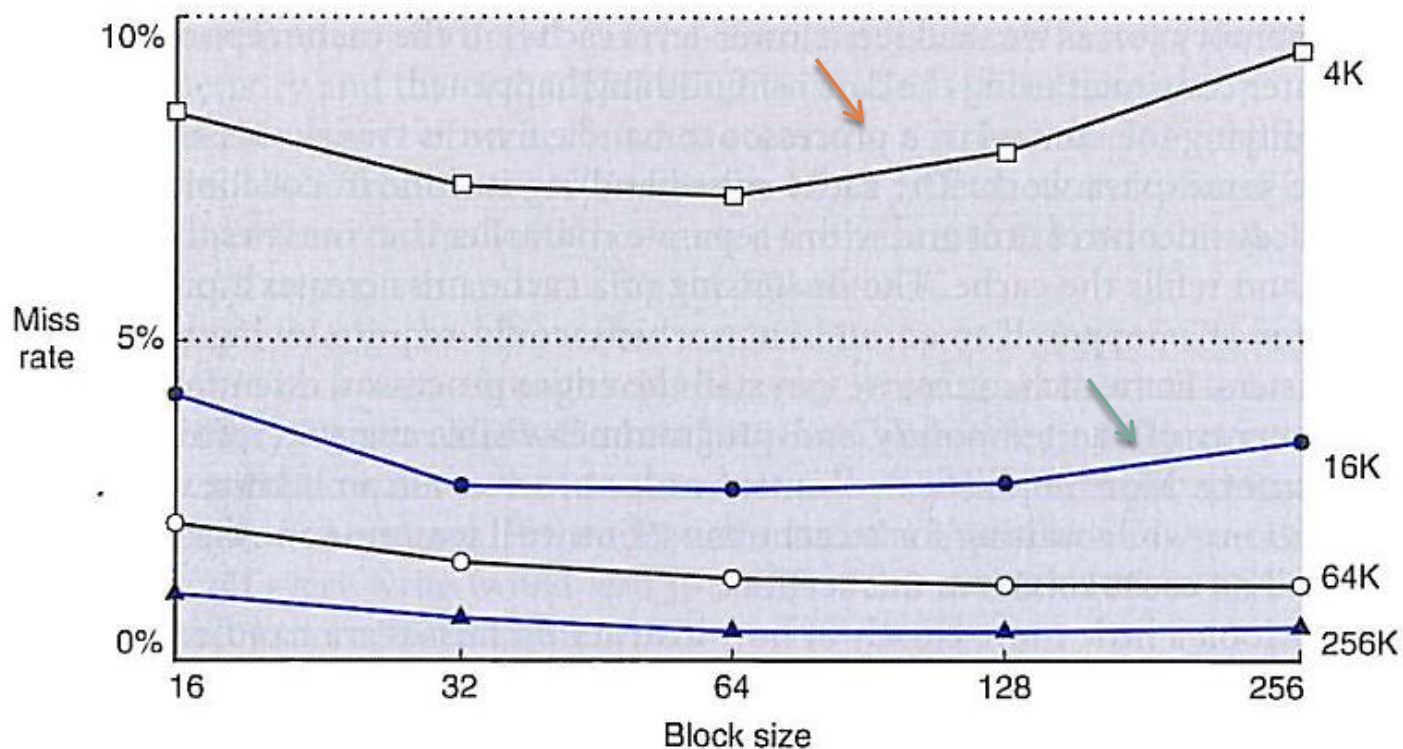
- ❑ Blocos maiores exploram a localidade espacial de forma a diminuir a taxa de erros (ou aumentar a taxa de acertos).



5.2 – O básico sobre caches

33

- Porém, a taxa de erros pode voltar a subir se o tamanho do bloco se torna uma fração significativa do tamanho da cache.



5.2 – O básico sobre caches

34

- ❑ O motivo disso é a diminuição do número de blocos que podem ser mantidos na cache.
- ❑ Assim, haverá competição entre os blocos para permanecer na cache.
- ❑ Como resultado, um bloco pode ser retirado da cache antes de muitas de suas palavras serem acessadas.
- ❑ Em outras palavras, a localidade espacial entre as palavras de um bloco diminui em blocos muito grandes.

5.2 – O básico sobre caches

35

- Outro problema é que simplesmente aumentar o tamanho do bloco irá acarretar um aumento no custo de um erro (*cache miss*).
- A penalidade em um *cache miss* é determinada pelo:
 - ▣ tempo para procurar o bloco no nível mais próximo da hierarquia de memória e trazê-lo para a cache.
- Esse tempo por sua vez é dividido em:
 - ▣ A latência para acessar a primeira palavra;
 - ▣ E o tempo de transferência para o resto do bloco.

5.2 – O básico sobre caches

36

- Maneiras de diminuir a penalidade:
- Recomeço antecipado (*early restart*)
 - ▣ Consiste em reiniciar o processamento assim que a palavra requerida estiver disponível, sem esperar a transferência do bloco inteiro. Funciona bem para cache de instruções (acesso mais previsível, grande parte feita de forma sequencial).
- Palavra requisitada primeiro (*requested word first*):
 - ▣ Outra forma, é retornar a palavra requisitada antes das demais palavras do bloco. As demais palavras são transferidas a partir do endereço posterior ao da palavra requisitada, retornando ao começo do bloco se necessário.

5.2 – O básico sobre caches

37

- **Lidando com erros (*cache misses*) (leituras):**
- O controle deve realizar os seguintes passos: 
 1. Enviar para a memória o endereço requerido (PC - 4);
 2. Instruir a memória para realizar uma leitura e esperar a memória completar o acesso;
 3. Escrever a informação na cache:
 1. Colocar os dados vindos da memória na porção de dados da linha da cache;
 2. Escrever os bits mais significativos do endereço no campo de tag;
 3. E colocar o bit de validade em 1.
 4. Recomeçar a execução da instrução corrente do início, a qual irá buscar novamente a instrução encontrando-a na cache desta vez.

5.2 – O básico sobre caches

38

- Lidando com escritas (*writes*):
 - ▣ Suponha que em uma instrução *store*, os dados são escritos apenas na cache (sem mudar a MP).
 - ▣ Nesse caso, MP e cache estarão em um estado ***inconsistente***.
 - ▣ Uma maneira de evitar isso é a **escrita direta** (*write-through*)
 - Que consiste em sempre escrever na cache e na MP.
 - ▣ Apesar de tratar escritas de forma simples esse esquema não provê uma boa performance.

5.2 – O básico sobre caches

39

- Observe que toda escrita requer uma escrita na MP também. Isso faz com que as escritas tomem um longo tempo.
- Por exemplo suponha que 10% das instruções são *stores*. Se a CPI sem erros de cache for 1, e se gastarmos 100 ciclos extras em cada escrita, teremos:

$$\text{CPI} = 1 + 100 \times 0,1 = 11.$$

- O que representa uma redução de performance por um fator maior que 10.

5.2 – O básico sobre caches

40

- ▣ Uma solução para esse problema é usar um **buffer de escrita** (*write buffer*).
 - O *buffer* de escrita armazena os dados enquanto eles esperam para ser escritos na MP.
 - Após os dados serem gravados na cache e no *buffer* o processador pode continuar a execução. Quando a escrita na MP é completada, o espaço deles no *buffer* é liberado.

5.2 – O básico sobre caches

41

- Uma alternativa à escrita direta é a:
 - ▣ **Escrita de volta** (*write-back* ou *copy-back*)
 - Nesse caso a escrita ocorre apenas na cache.
 - Os dados são escritos na MP apenas quando o bloco que foi modificado na cache for substituído.

5.2 – O básico sobre caches

42

- Projetando o sistema de memória para suportar caches
 - O processador é tradicionalmente ligado à memória por meio de um barramento.
 - A frequência do barramento é usualmente bem menor que a do processador.
 - A velocidade do barramento afeta a penalidade de erro da cache.

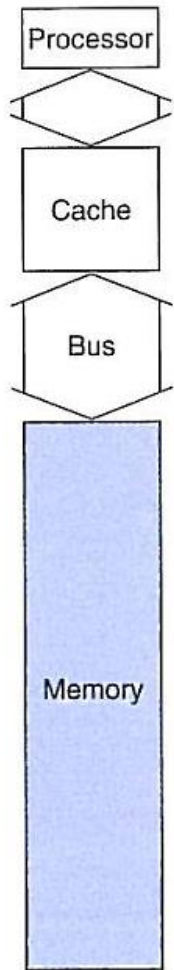
5.2 – O básico sobre caches

43

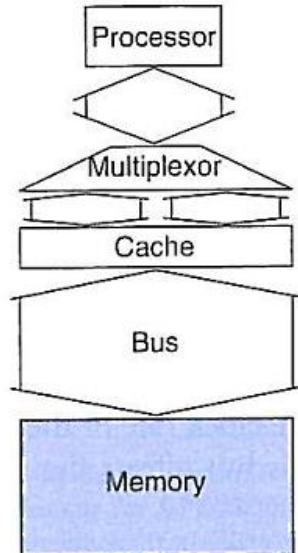
- Para entendermos melhor suponha:
 - 1 ciclo de barramento para enviar o endereço;
 - 15 ciclos de barramento para cada acesso a DRAM iniciar;
 - 1 ciclo de barramento para enviar uma palavra de dados.
- Dessa forma, se um bloco da cache tem 4 palavras e a memória tem largura de uma palavra, temos:
 - $1 + (4 \times 15) + (4 \times 1) = 65$ ciclos de barramento.
 - O que nos dá um número de bytes por ciclo de barramento igual a $(4 \text{ palavras} \times 4 \text{ bytes}) / 65 = 0,25 \text{ bytes/ciclo barr.}$
No caso de um erro na cache.

5.2 – O básico sobre caches

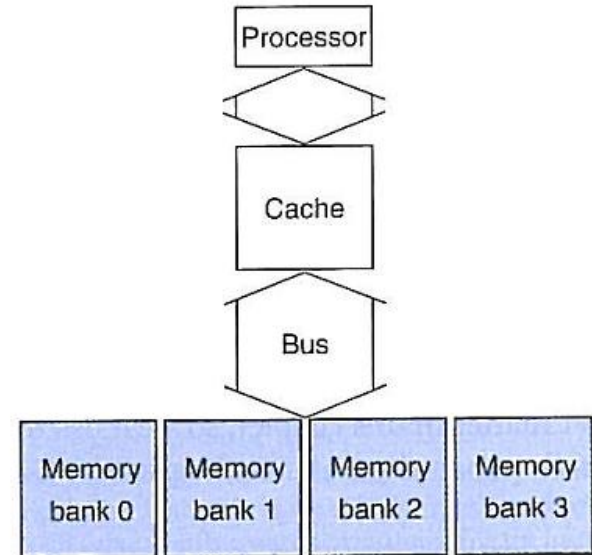
44



a. One-word-wide memory organization



b. Wider memory organization



c. Interleaved memory organization

5.2 – O básico sobre caches

45

- Para o caso a) já calculamos 65 ciclos de barramento.
- Para o caso b) memória e barramento largos
 - ▣ Se uma locação de memória armazena 2 palavras e o barramento pode transmitir 2 palavras por vez, temos:
 - ▣ $1 + (2 \times 15) + (2 \times 1) = 33$ ciclos de barramento.
 - ▣ Nesse caso a melhoria se dá tanto no tempo de acesso quanto no tempo de transferência dos dados.
 - ▣ Para um erro na cache a largura de banda é de:
 $(4 \text{ palavras} \times 4 \text{ bytes}) / 33 = 0,48 \text{ bytes/ciclo de barr.}$

5.2 – O básico sobre caches

46

- Para o caso c) organização em bancos
 - ▣ Nesse caso, a memória é organizada em bancos, de forma a permitir ler e escrever múltiplas palavras em um único tempo de acesso.
 - ▣ Cada banco pode ter largura e uma palavra para evitar mudanças no barramento.
 - ▣ Se temos 4 bancos, enviando 1 endereço para vários bancos obtemos:
 - ▣ $1 + (1 \times 15) + (4 \times 1) = 20$ ciclos de barramento.
 - ▣ Para um erro na cache a largura de banda é de:
 $(4 \text{ palavras} \times 4 \text{ bytes}) / 20 = 0,8 \text{ bytes/ciclo de barr.}$

5.3 – Medindo e melhorando a performance da cache

47

- Vamos agora examinar alguns meios de medir a performance da cache;
- Bem como, estudar duas maneiras de melhorar essa performance:
 - ▣ A primeira, foca na redução da taxa de erros por meio da redução da probabilidade de dois blocos de memória diferentes, concorrerem pela mesma posição na cache.
 - ▣ A segunda, basea-se em reduzir a penalidade do erro na cache (*miss penalty*), por meio da adição de novos níveis à hierarquia.
 - Essa técnica é conhecida como cache multinível (*multilevel cache*).

5.3 – Medindo e melhorando a performance da cache

48

- O tempo de CPU pode ser dividido:
 - ▣ Nos ciclos de clock que a CPU gasta executando o programa;
 - ▣ E nos ciclos de clock que a CPU gasta esperando o sistema de memória.

$$t_{CPU} = (\text{Ciclos de CPU de execução} + \text{Ciclos de CPU de espera}) \times T_{ciclo}$$

- aqui assumiremos que os ciclos de CPU de espera (ou parada) são oriundos principalmente de erros na cache.

5.3 – Medindo e melhorando a performance da cache

49

- os ciclos de CPU de espera podem ser divididos em provocados por escritas e por leituras, assim:

Ciclos de CPU de espera = Ciclos de espera por leitura + Ciclos de espera por escrita

- Os ciclos de espera por leitura, são dados por:

$$\text{Ciclos de espera por leitura} = \frac{\text{leituras}}{\text{programa}} \times \text{Taxa de erros de leitura} \times \text{Penalidade de erros de leitura}$$

- Os ciclos de espera por escrita:

$$\begin{aligned} \text{Ciclos de espera por escrita} = & \left(\frac{\text{escritas}}{\text{programa}} \times \text{Taxa de erros de escrita} \times \text{Penalidade de erros de escrita} \right) \\ & + \text{espera pelo buffer de escrita} \end{aligned}$$

5.3 – Medindo e melhorando a performance da cache

50

- Podemos assumir que em um sistema bem projetado a espera pelo buffer de escrita é desprezível.
- Podemos também considerar que as penalidades de escrita e leitura são iguais ao tempo de buscar um bloco na memória e assim:

$$\text{Ciclos de espera por memória} = \frac{\text{acessos à memória}}{\text{programa}} \times \text{Taxa de erros} \times \text{Penalidade de erro}$$

- Ainda podemos expressar, em função do número de instruções:

$$\text{Ciclos de espera por memória} = \frac{\text{Instruções}}{\text{programa}} \times \frac{\text{Erros}}{\text{Instrução}} \times \text{Penalidade de erro}$$

5.3 – Medindo e melhorando a performance da cache

51

- Exemplo: calculando a performance de cache
 - ▣ Assuma que a taxa de erros de uma cache de instruções é 2% e que a taxa de erros de uma cache de dados é 4%.
 - ▣ Se um processador tem uma CPI igual a 2, sem considerar esperas por memória, e uma penalidade de 100 ciclos, para qualquer erro.
 - ▣ Determine o quanto mais rápido esse processador seria se tivesse uma cache perfeita (nunca ocorre erros).
 - ▣ Assuma que a frequência de *loads* e *stores* é de 36% no programa.

5.3 – Medindo e melhorando a performance da cache

52

- Solução:
- Em termos do número de instruções, temos que:

$$\text{ciclos de erro de busca de instrução} = I \times 0,02 \times 100 = 2I$$

- O programa é 100% instruções, uma taxa de erro de 2% em busca de instruções é aplicada a totalidade das instruções do programa.

- Para a parcela de dados, temos:

$$\text{ciclos de erro de busca de dados} = I \times 0,36 \times 0,04 \times 100 = 1,44I$$

5.3 – Medindo e melhorando a performance da cache

53

- Assim os ciclos de espera por memória, são no total:

$$\text{ciclos de espera por memória} = 2I + 1,44I = 3,44I$$

- O que dá mais de 3 ciclos parados por instrução. Dessa forma consumimos com espera, por instrução:

$$CPI_{\text{com espera}} = 2 + 1,44 = 3,44 \text{ ciclos/inst.}$$

- A CPI total considerando espera é:

$$\begin{aligned} CPI_{\text{total}} &= CPI_{\text{sem espera}} + CPI_{\text{com espera}} \\ &= 2 + 3,44 = 5,44 \text{ ciclos/inst.} \end{aligned}$$

5.3 – Medindo e melhorando a performance da cache

54

- Comparando então os tempos de CPU, obtemos:

$$\frac{P_{sem\ espera}}{P_{com\ espera}} = \frac{t_{CPU\ com\ espera}}{t_{CPU\ sem\ espera}} = \frac{I \times CPI_{com\ espera} \times T_{ciclo}}{I \times CPI_{sem\ espera} \times T_{ciclo}} = \frac{CPI_{com\ espera}}{CPI_{sem\ espera}} = \frac{5.44}{2.00} = 2.72$$

- Se diminuíssemos a $CPI_{sem\ parada}$ para 1, sem mudar a frequência, teríamos:

Porém o tempo de execução gasto com espera:

$$\frac{P_{sem\ espera}}{P_{com\ espera}} = \frac{CPI_{com\ espera}}{CPI_{sem\ espera}} = \frac{4.44}{1.00} = 4.44$$

$$T_{CPU\ gasto\ apenas\ com\ espera} = \frac{3.44}{5.44} = 0,63 = 63\%$$

$$T_{CPU\ gasto\ apenas\ com\ espera} = \frac{3.44}{4.44} = 0,77 = 77\%$$

5.3 – Medindo e melhorando a performance da cache

55

- Tempo médio de acesso a memória (AMAT – Average Memory Access Time)

$$AMAT = t_{acerto} + taxa\ de\ erros \times penalidade\ de\ erro$$

- Exemplo: considere um processador com $T_{ciclo} = 1ns$, uma taxa de erros 0,05 e um tempo de acesso a cache (incluindo a detecção de acerto) de 1 ciclo de clock e uma penalidade de erro de 20 ciclos. Calcule o AMAT.

$$\begin{aligned} AMAT &= 1 + 0,05 \times 20 = 2\ ciclos \\ &= 2\ ciclos \times T_{ciclo} = 2 \times 1ns = 2ns \end{aligned}$$

5.3 – Medindo e melhorando a performance da cache

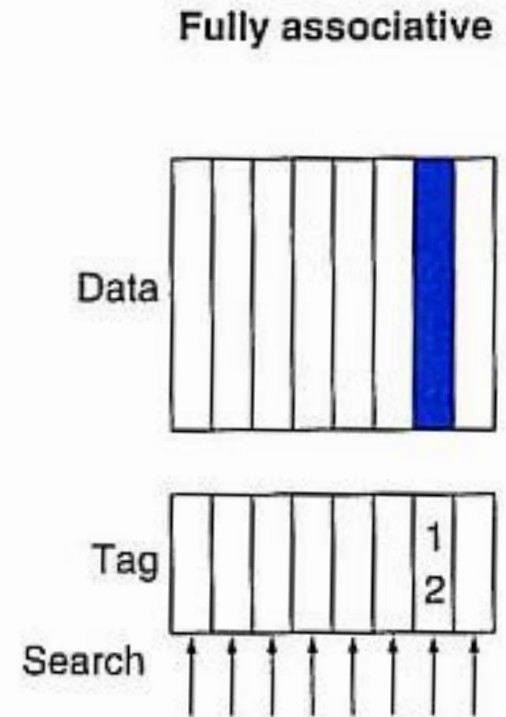
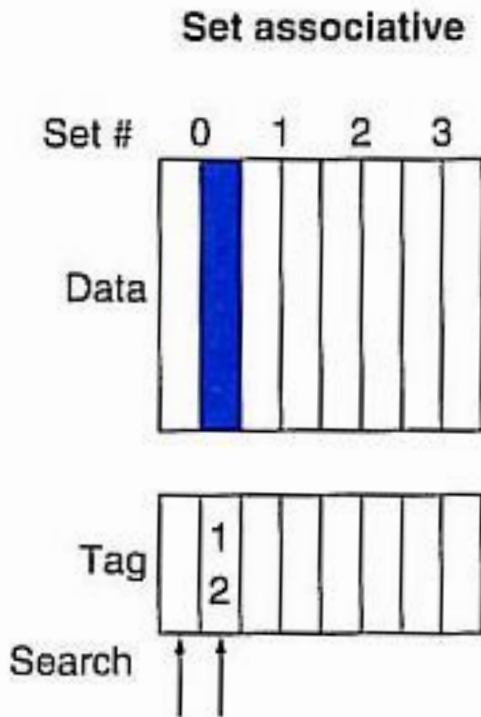
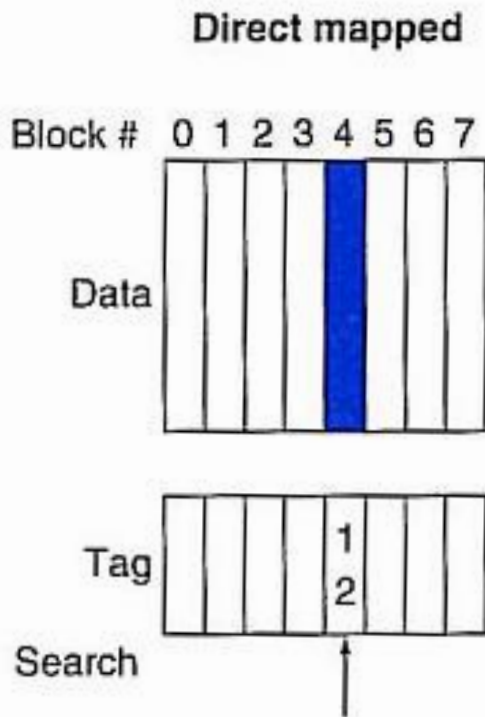
56

- **Redução da taxa de erros por meio de uma localização mais flexível dos blocos na cache.**
 - ▣ Mapeamento totalmente associativo
 - Qualquer bloco da memória pode ser guardado em qualquer posição da cache.
 - ▣ Mapeamento associativo por conjuntos
 - Um bloco da memória pode ser guardado em qualquer posição dentro de um determinado conjunto da cache.
 - ▣ Conjunto
 - Determinadas posições da cache (duas no mínimo), nas quais um bloco de memória pode ser mapeado.

5.3 – Medindo e melhorando a performance da cache

57

- **Mapeamentos: Direto, associativo por conjuntos e totalmente associativo**



5.3 – Medindo e melhorando a performance da cache

58

- No mapeamento associativo por conjuntos
 - ▣ **Conjunto = (endereço de bloco) módulo (número de conjuntos da cache)**
 - ▣ Como um bloco pode ser colocado em qualquer posição do conjunto todas os blocos dentro de um conjunto devem ser verificados para determinar um acerto na cache.

59

Eight-way set associative (fully associative)

5.3 – Medindo e melhorando a performance da cache

60

□ Exemplo:

- ▣ Assuma que existem 3 pequenas caches, cada uma contendo 4 blocos de uma palavra.
- ▣ Uma delas é totalmente associativa;
- ▣ A segunda é associativa por conjuntos com duas posições por conjunto;
- ▣ E a terceira possui mapeamento direto.
- ▣ Encontre o números de erros para cada organização de cache, dada a seguinte sequência de endereços: 0, 8, 0, 6 e 8.

5.3 – Medindo e melhorando a performance da cache

61

- Para a cache que usa mapeamento direto, temos:

Block address	Cache block
0	$(0 \text{ modulo } 4) = 0$
6	$(6 \text{ modulo } 4) = 2$
8	$(8 \text{ modulo } 4) = 0$

- Acompanhando o estado da cache, após cada solicitação de endereço: 0, 8, 0, 6 e 8, temos:

Address of memory block accessed	Hit or miss	Contents of cache blocks after reference			
		0	1	2	3
0	miss	Memory[0]			
8	miss	Memory[8]			
0	miss	Memory[0]			
6	miss	Memory[0]		Memory[6]	
8	miss	Memory[8]		Memory[6]	

5.3 – Medindo e melhorando a performance da cache

62

- Para a cache associativa por conjuntos, temos:

Block address	Cache set
0	$(0 \text{ modulo } 2) = 0$
6	$(6 \text{ modulo } 2) = 0$
8	$(8 \text{ modulo } 2) = 0$

- Antes de acompanhar o estado da cache, temos que determinar uma política de substituição de bloco.
- Já que teremos que escolher qual bloco dentro de um conjunto deve ser substituído.

5.3 – Medindo e melhorando a performance da cache

63

- Usando a política do menos recentemente utilizado, ou seja o bloco dentro do conjunto que foi usado a mais tempo, temos:

Address of memory block accessed	Hit or miss	Contents of cache blocks after reference			
		Set 0	Set 0	Set 1	Set 1
0	miss	Memory[0]			
8	miss	Memory[0]	Memory[8]		
0	hit	Memory[0]	Memory[8]		
6	miss	Memory[0]	Memory[6]		
8	miss	Memory[8]	Memory[6]		

- Tivemos 4 erros.

5.3 – Medindo e melhorando a performance da cache

64

- A cache totalmente associativa pode ser pensada como tendo um único conjunto de 4 lugares (blocos). E um bloco de memória pode se colocado em qualquer desses lugares, assim:

Address of memory block accessed	Hit or miss	Contents of cache blocks after reference			
		Block 0	Block 1	Block 2	Block 3
0	miss	Memory[0]			
8	miss	Memory[0]	Memory[8]		
0	hit	Memory[0]	Memory[8]		
6	miss	Memory[0]	Memory[8]	Memory[6]	
8	hit	Memory[0]	Memory[8]	Memory[6]	

- Melhor performance 3 erros apenas.

Localizando um bloco na cache (mapeamento associativo por conjuntos)

